

HOPE LEARNING

User Based Recommendation System: Technical Writeup

(by Dr. N. Dilip Raja)

This document provides a comprehensive explanation of the User-Based Recommendation System code, which implements collaborative filtering to predict movie ratings and generate personalized recommendations based on user similarity.

File Information

- **Source File:** MultipleFiles/1 User Based Recommendation System.py
- **Purpose:** User-Based Collaborative Filtering for Movie Recommendations
- **Dataset:** MovieLens 100k

Step-by-Step Processes

1. Import Required Libraries

The code begins by importing essential Python libraries for data manipulation and computation:

- **pandas:** Used for data frame operations and handling structured data
- **numpy:** Used for numerical computations and array manipulations
- **sklearn.metrics.pairwise:** Provides the `pairwise_distances` function for calculating cosine similarity
- **itertools:** Used for flattening nested lists

2. Load Dataset

The system loads the MovieLens 100k dataset, which contains user movie ratings:

- **File:** `ml-100k/u.data`
- **Separator:** Tab-delimited file (`\t`)
- **Column Names:** `User_id`, `Movie_id`, `Ratings`, `Unique_timestamp`
- **Encoding:** Latin-1 encoding

The dataset is loaded into a pandas DataFrame for processing.

3. Identify Number of Users

The code determines the total number of unique users in the dataset:

- Uses `Ratings.User_id.unique().shape[0]` to count distinct users

This information is crucial for setting up the similarity matrix

4. Create Pivot Table

A user-movie rating matrix (pivot table) is created:

- **Index:** User_id
- **Columns:** Movie_id
- **Values:** Ratings

This creates a sparse matrix where rows represent users and columns represent movies they have rated

5. Data Preprocessing

The pivot table undergoes preprocessing:

- **NaN Replacement:** Missing values (NaN) are replaced with 0 using `Pivot_table.replace(np.nan, 0)`

This ensures the similarity calculations work properly without missing value errors

6. Calculate User Similarity

The code computes cosine similarity between all users:

- Uses `sklearn.metrics.pairwise_distances()` with `metric='cosine'`

The resulting similarity matrix shows how similar each pair of users is based on their rating patterns. Lower cosine distance indicates higher similarity

7. Rating Prediction Function

A prediction function is defined to forecast ratings:

- **Formula:** `User_ratings_mean + Similarity.dot(Ratings_diff) / np.abs(Similarity).sum(axis=1)`

Takes into account the user's mean rating and adjusts based on similar users' ratings. Returns a prediction matrix for all user-movie pairs

8. Generate Predictions

The prediction function is executed:

- Creates a **User_prediction** matrix containing predicted ratings for all users

This matrix is used to recommend highly rated movies to users

9. Find Similar Users

For a specific input user (User ID 34 in this case):

The code sorts the similarity values to find the top 5 most similar users

Uses the cosine similarity table to identify users with closest rating patterns

10. Extract Movies Watched by Similar Users

The code collects all movies watched by similar users:

Iterates through each similar user

Retrieves their Movie_id from the ratings table

Combines all movie lists into a single list using `itertools.chain.from_iterable()`

11. Filter Recommended Movies

The code filters out movies the input user has already watched:

- Creates a list of movies the input user has already seen: **Input_User_Watched_MovieID**

Compares with movies watched by similar users

Only keeps movies the input user hasn't watched yet

12. Select Highest Rated Predictions

The system predicts which movies the user would rate highly:

Uses the prediction matrix to find predicted ratings for recommended movies

Filters movies with predicted ratings ≥ 1

These become the final recommended movies

13. Load Movie Information

The code loads movie metadata:

- **File:** `ml-100k/u.item`
- **Contains:** Movie id, title, release date, genres, and other metadata

Used to map movie IDs to readable movie titles

14. Generate Final Movie List

The final step creates a list of recommended movie titles:

Maps each recommended Movie ID to its title from the items file

Outputs the final list of personalized movie recommendations

Summary

This User-Based Recommendation System successfully:

1. Loads and preprocesses the MovieLens 100k rating data
2. Computes user similarity using cosine distance
3. Predicts ratings for user-movie pairs
4. Identifies the most similar users
5. Generates personalized movie recommendations based on what similar users have watched and rated highly

The system provides targeted recommendations that the user is likely to enjoy based on the preferences of users with similar rating behaviors.
